

EchoQuality 2.0 Localhost

Diseño, implementacion, operacion y defensa técnica

Arquitectura jerárquica de reconocimiento de patrones para calidad ecocardiográfica y procesamiento adaptativo

Curso	SP-2155 Reconocimiento de Patrones y Sistemas Inteligentes
Docente	Sergio Moya Valerin
Institucion	Universidad de Costa Rica
Versión del software	EchoQuality 2.0.1
Alcance	Servidor local para entrenamiento, inferencia, diagnóstico y procesamiento adaptativo

Prototipo academico. No es un dispositivo medico ni debe utilizarse para decisiónes clínicas.

2026-10-11

Resumen ejecutivo

Este documento explica el desarrollo de un localhost para EchoQuality 2.0. La interfaz permite selecciónar un caso de la base CAMUS o cargar archivos, ejecutar la extraccion jerárquica de patrones, aplicar el Quality Gate con fusión tardia, comparar el caso contra Good References y evaluar procesamiento adaptativo condicionado al diagnóstico.

La solucion usa Streamlit como servidor de interfaz, pero conserva el nucleo científico dentro del paquete Python. Esta separacion evita que la logica del algoritmo quede atrapada en la interfaz y facilita las pruebas, la reutilizacion y una futura migracion hacia una API o un servicio institucional.

El servidor se enlaza de forma predeterminada a 127.0.0.1:8501. Esto significa que la aplicación se ejecuta en la computadora del investigador y no se publica automaticamente en Internet ni en la red institucional.

1. Objetivo y alcance

El objetivo del localhost es convertir EchoQuality 2.0 en una herramienta demostrable y operable desde un navegador, manteniendo la trazabilidad del pipeline. El usuario puede ejecutar dos tipos de proceso: entrenamiento offline mediante un YAML y análisis online de un caso mediante artefactos ya entrenados.

- Entrenamiento: construye el manifest, extrae features, valida modelos, genera Good References y guarda artefactos.
- Inferencia: carga esos artefactos y analiza un nuevo caso sin reentrenar el modelo.
- Diagnóstico: identifica el nivel Global, Anatomico o Textural que presenta mayor riesgo o desviacion.
- Procesamiento adaptativo: evalua transformaciónes solamente cuando el diagnóstico lo justifica.

El localhost no convierte el prototipo en un dispositivo medico. Su funcion es investigativa: demostrar el flujo, ejecutar experimentos reproducibles, revisar resultados y apoyar la defensa metodológica ante el profesor.

2. Arquitectura general del localhost

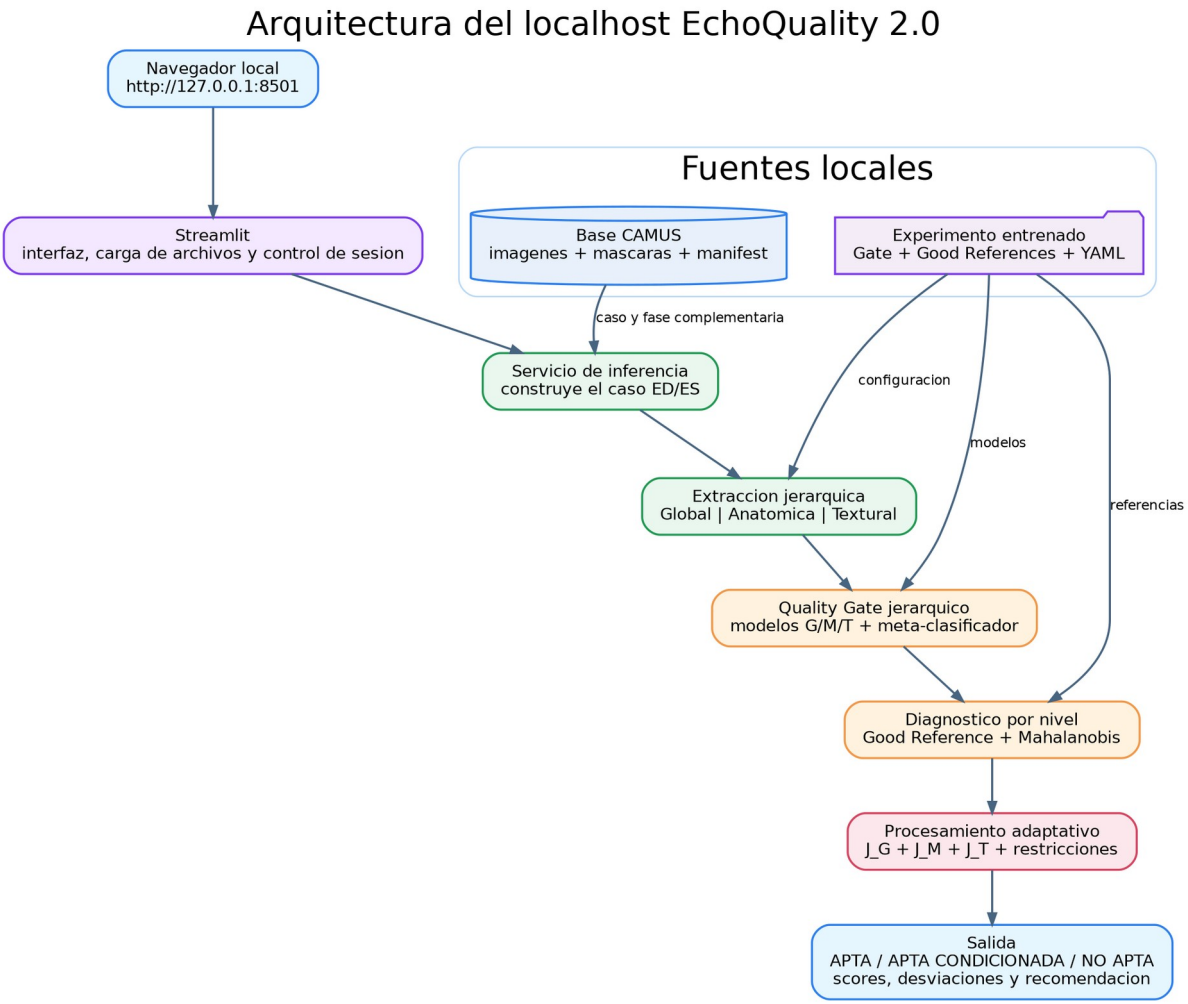


Figura 1. El navegador controla una aplicación Streamlit local que consume la base CAMUS y los artefactos de un experimento entrenado.

La interfaz no contiene el algoritmo completo en un solo archivo. streamlit_app.py se ocupa de widgets, visualizacion y control de sesión; inference.py implementa la preparacion del caso, la extraccion, la prediccion, el diagnóstico y el guardado de resultados; los módulos de analysis, recognition y optimization mantienen la logica científica.

3. Separacion entre entrenamiento e inferencia

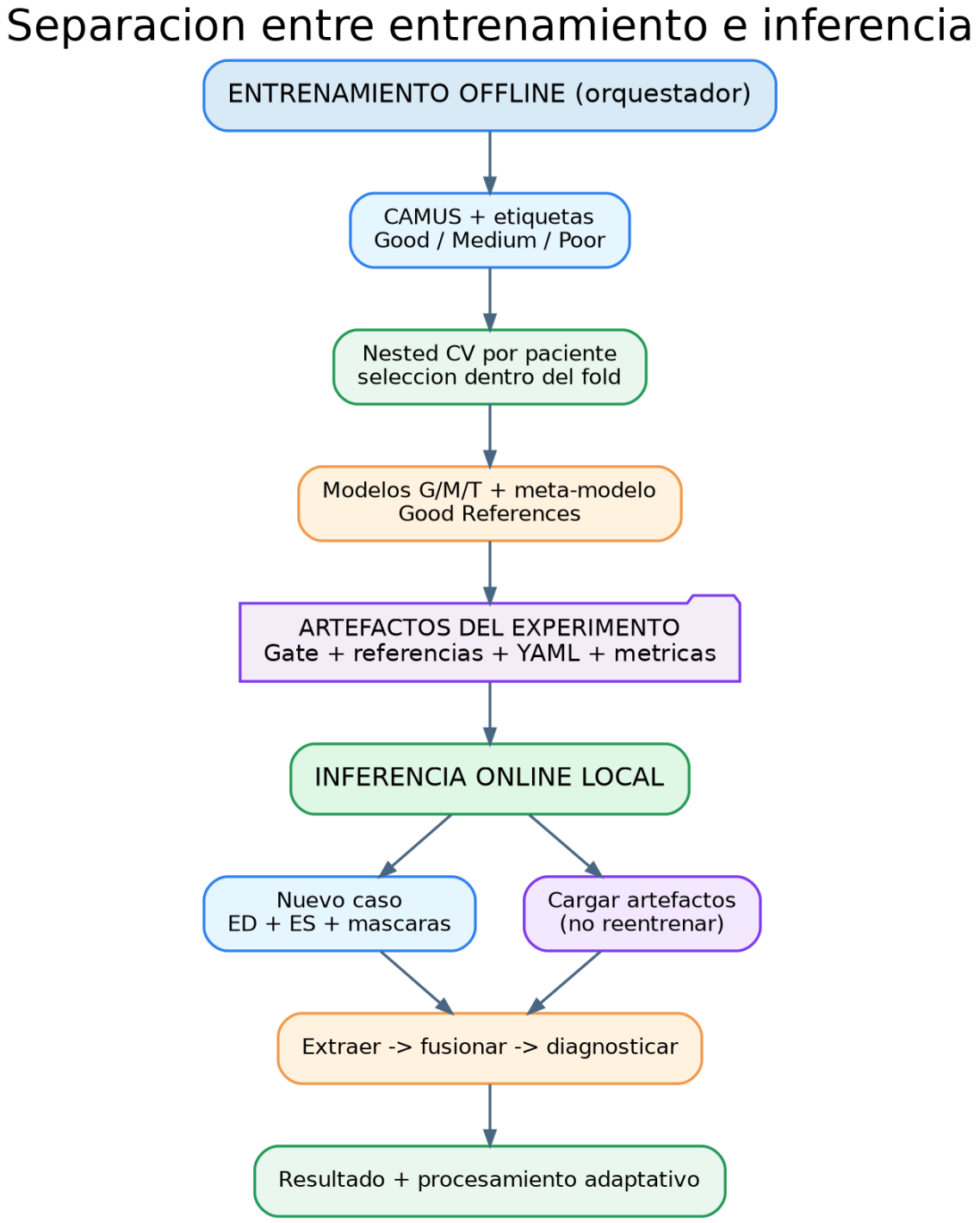


Figura 2. Los modelos se entrenan offline y se reutilizan durante la inferencia. Una imagen nueva no debe provocar un reentrenamiento automatico.

Esta separacion es esencial para evitar confundir dos preguntas distintas. El entrenamiento estima parámetros usando una cohorte con etiquetas y nested cross-validation; la inferencia aplica esos parámetros a un caso nuevo. Si el modelo se reentrenara con cada imagen, la decisión dejaria de ser comparable y reproducible.

4. Requisitos de entrada

El Quality Gate actual opera a nivel paciente-vista. Durante el entrenamiento, las features de ED y ES se agregan con mean y abs_delta. Por esta razon, el algoritmo completo necesita las dos fases y sus máscaras:

```
ED image + ED mask
ES image + ES mask
```

Las máscaras CAMUS codifican LV=1, MYO=2 y LA=3. La máscara es necesaria para la representación anatómica y para restringir la textura a la ROI miocardica. Sin máscara, la representación global puede calcularse, pero Anatomia y Textura pierden el soporte que les da significado.

El localhost ofrece tres modos: seleccionar un paciente-vista desde el manifest; cargar ED y ES con sus máscaras; o cargar una sola imagen en modo exploratorio. En este ultimo caso se calculan features y distancias a Good Reference de frame, pero no se emite la decisión final del gate entrenado con ED y ES.

5. Flujo matemático del análisis de un caso

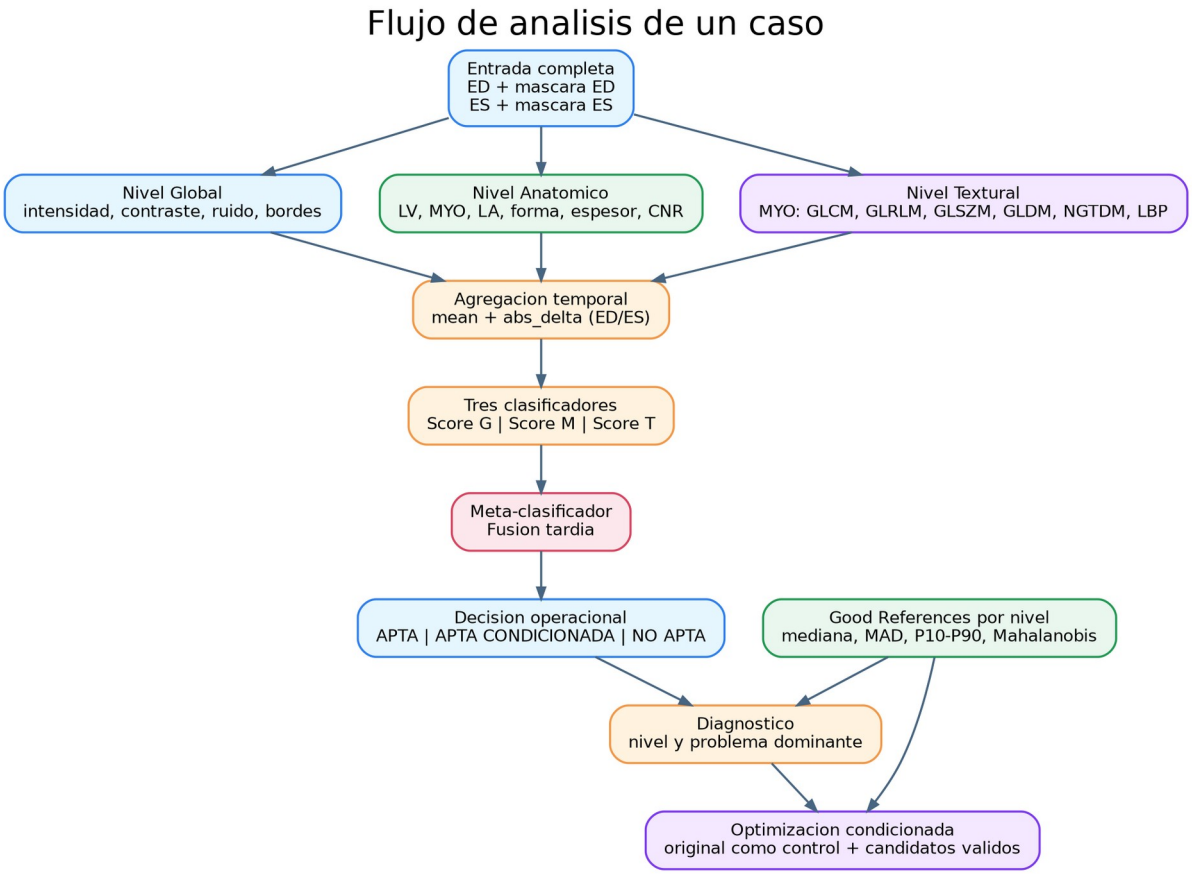


Figura 3. Flujo completo desde ED/ES hasta la decisión y la optimización condicionada.

5.1 Representaciónes por nivel

Para cada fase se extraen tres vectores independientes:

```
x_G = global features
x_M = anatomical features
x_T = myocardial texture features
```

Global resume intensidad, contraste, nitidez, ruido, soporte y saturacion. Anatomia usa LV, MYO y LA para medir forma, cobertura, espesor, fuerza de borde y CNR. Textura se calcula principalmente dentro del miocardio e incluye GLCM, GLRLM, GLSZM, GLDM, NGTDM y LBP.

5.2 Agregacion ED/ES

Para cada feature f se construyen, como minimo:

```
f__mean = (f_ED + f_ES) / 2
f__abs_delta = |f_ED - f_ES|
```

La media resume la vista y abs_delta conserva evidencia de desacuerdo entre fases. Esto mejora la versión anterior, en la que view_features era solamente la media y podia ocultar un defecto presente en una sola fase.

5.3 Clasificadores de nivel y fusión tardia

Cada nivel produce una probabilidad de NO APTA:

```
s_G = P(y=1 | x_G)
s_M = P(y=1 | x_M)
s_T = P(y=1 | x_T)
```

El meta-clasificador recibe s = [s_G, s_M, s_T] y calcula la probabilidad final:

```
p_final = P(y=1 | s_G, s_M, s_T)
```

Los modelos permitidos siguen siendo Logistic Regression y Random Forest. La selección supervisada de features y los hiperparámetros se ajustan dentro de los folds de entrenamiento, agrupando por patient_id para evitar fuga de información.

5.4 Umbral y estados operacionales

La decisión binaria primaria utiliza un umbral tau selecciónado bajo una restriccion minima de sensibilidad para NO APTA:

```
NO APTA if p_final >= tau
APTA otherwise
```

EchoQuality agrega un estado operacional intermedio. Si p_final no supera el umbral, pero un nivel presenta riesgo alto o existe una alarma respecto del Good Reference, el caso se clasifica como APTA CONDICIONADA. Esto no es una tercera clase aprendida; es una regla operacional explicable.

6. Good Reference y diagnóstico por nivel

Los Good References se construyen con observaciones Good y se separan por nivel y vista. Cada referencia almacena mediana, MAD escalado, percentiles P10/P90 y una covarianza regularizada con Ledoit-Wolf.

6.1 Z robusto

```
z_j = (x_j - median_j) / (1.4826 * MAD_j)
```

El z robusto identifica features por encima o por debajo del patron Good sin depender excesivamente de valores extremos.

6.2 Distancia de Mahalanobis

```
D_M(x) = sqrt( z^T * Precision * z )
```

La distancia combina varias features y considera sus correlaciones. La alarma se activa cuando la distancia supera el percentil 95 de las distancias observadas en el grupo Good de referencia.

6.3 Nivel y problema dominante

El nivel dominante se obtiene del mayor score de NO APTA entre Global, Anatomico y Textural. El problema dominante se obtiene de las desviaciones robustas de magnitud relevante, por ejemplo brillo/contraste, ruido/speckle, bordes, geometría anatómica, dropout o textura.

7. Procesamiento adaptativo

El procesamiento no se aplica indiscriminadamente. El diagnóstico selecciónа familias de transformaciónes y la imagen original se incluye siempre como candidato identity.

Problema	Familia evaluada	Precaucion
Brillo o contraste	Gamma, stretch, CLAHE controlado	Puede cambiar la distribucion de textura
Ruido o speckle	Bilateral, mediana, difusión anisotropica	Puede borrar microtextura
Bordes debiles	Unsharp conservador	Puede amplificar speckle y producir halos
Saturacion	Revisión de compresion/contraste	La información recortada no se recupera
Dropout o anatomia truncada	Revisión o readquisicion	No se debe inventar anatomia ausente

7.1 Funcion objetivo

```
J = w_G * J_G + w_M * J_M + w_T * J_T + w_S * SSIM - penalties
```

J_G mide la mejora de distancia global; J_M premia preservacion anatómica; J_T combina mejora textural y baja deriva; SSIM actua como fidelidad global. Ademas se exigen restricciones de edge F1, deriva anatómica, deriva textural y clipping.

Si ningun candidato valido supera a identity por la mejora minima requerida, la recomendacion es MANTENER ORIGINAL. Esta regla evita que el sistema aplique un filtro solo por apariencia visual.

8. Dos ramas de imagen

La imagen usada para encontrar el miocardio no necesariamente debe ser la misma versión usada para medir su textura. EchoQuality separa conceptualmente:

```
Rama A: image -> enhancement optional -> segmentation -> mask
Rama B: controlled/original image + mask -> texture measurement
```

La rama A puede usar realce para localizar contornos. La rama B conserva una versión definida a priori para evitar que CLAHE, interpolacion o denoising alteren el fenomeno textural que se desea medir. Ambas ramas comparten la máscara final.

9. Diseno del software

Componente	Responsabilidad
app/streamlit_app.py	Interfaz, carga de archivos, visualizacion, descargas y control del orquestador.
application/inference.py	Prepara casos, extrae features, predice, diagnostica, optimiza y guarda resultados.
analysis/	Convierte imágenes y máscaras en representaciónes cuantitativas.
recognition/	Modelos de nivel, meta-clasificador, Good Reference y diagnóstico.
processing/	Transformaciones puntuales, espaciales y preservadoras de

	bordes.
optimization/	Funcion objetivo, restricciones y busqueda de candidatos.
experiments/	Orquestador, etapas, auditorias y rutas por experimento.
configs/	YAML reproducibles con dataset, seed, features, CV y procesamiento.
results/	Artefactos y reportes generados; no se versiónan en GitHub.

La separacion entre interfaz y servicio de inferencia permite probar el algoritmo sin depender de Streamlit. Tambien deja abierta una futura migracion a FastAPI, una aplicación institucional o un servicio de laboratorio.

10. Funcionamiento de Streamlit

Streamlit ejecuta un script Python como servidor local y vuelve a ejecutar el script cuando cambia un widget. Por ello, los modelos y Good References se cargan mediante cache_resource, mientras que el resultado del caso se conserva en session_state. Los archivos cargados se leen desde un buffer y se materializan temporalmente cuando el lector de NIfTI o NPY necesita una ruta.

La configuración .streamlit/config.toml fija address=127.0.0.1, port=8501 y maxUploadSize=512. El limite puede ajustarse por proyecto sin modificar el código de la interfaz.

11. Orquestador y YAML

El orquestador ejecuta etapas con dependencias y registra el estado en experiment_state.json. Puede reanudar, forzar etapas o usar una configuración fast para pruebas de ingenieria.

```
python run_experiment.py --config configs\experiments\camus10.yaml --fast
```

El YAML separa decisiónes experimentales del código. Modificar n_patients, random seed, cuantizacion GLCM, numero de folds o restricciones de procesamiento no requiere reescribir el algoritmo. Cada configuración queda guardada junto con los resultados, facilitando reproducibilidad.

12. Seguridad, privacidad y alcance local

- El servidor se enlaza a 127.0.0.1 y no debe cambiarse a 0.0.0.0 sin revisión de seguridad institucional.
- CAMUS permanece fuera del repositorio y se referencia mediante manifest y CAMUS_ROOT.
- Los archivos cargados se materializan en un directorio temporal y se eliminan al finalizar el análisis.
- Los resultados solo se guardan en disco cuando el usuario lo solicita.
- No se incluye autenticacion porque el alcance es localhost de un solo investigador. Una publicacion en red exigiria autenticacion, cifrado, control de acceso, registro y evaluación de privacidad.

13. Instalacion paso a paso

1. Descomprimir o clonar echoquality-v2 en una carpeta permanente.
2. Abrir la carpeta desde GitHub Desktop y luego Repository -> Open in Visual Studio Code.
3. Crear o activar el entorno Conda echoquality-v2.

```
conda env create -f environment.yml
conda activate echoquality-v2
python -m pip install -e ".[app]"
```

4. Seleccionar el interprete echoquality-v2 en VS Code.
5. Configurar CAMUS_ROOT en Anaconda Prompt o PowerShell.
6. Ejecutar pruebas unitarias.

```
pytest
```

7. Ejecutar el experimento smoke y validar el localhost.

```
python run_experiment.py --config configs\experiments\smoke.yaml --force --fast
python scripts\13_validate_localhost.py --experiment-root results\smoke --config
configs\experiments\smoke.yaml
```

8. Iniciar el servidor.

```
python run_streamlit.py
```

9. Abrir http://127.0.0.1:8501 y selecciónar results/smoke o un experimento CAMUS completado.

14. Flujo de desarrollo con GitHub y VS Code

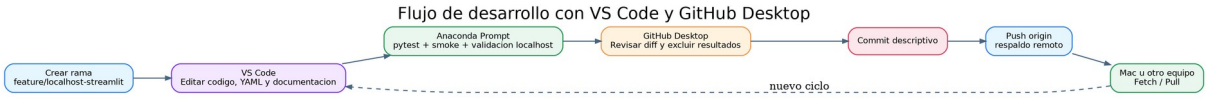


Figura 4. Ciclo recomendado de desarrollo, validación y publicacion.

La funcionalidad debe desarrollarse en una rama separada, probarse antes del commit y revisarse en GitHub Desktop. Los resultados, modelos, datos CAMUS y archivos temporales estan excluidos por .gitignore; el repositorio debe contener código, configuraciónes, pruebas y documentacion.

15. Validación del desarrollo

La entrega se valida en cuatro niveles:

Nivel	Prueba	Criterio
Sintaxis	compileall	Todos los módulos compilan.
Unitario	pytest	Carga de manifest, features, gate, referencias y procesamiento.
Integracion	smoke.yaml	Las ocho etapas finalizan y generan artefactos.
Inferencia	13_validate_localhost.py	Un paciente-vista produce scores, diagnóstico y salidas.
Interfaz	Streamlit local	Carga recursos, analiza casos y descarga reportes.

La validación final con CAMUS debe comparar las salidas de la interfaz contra una ejecución programatica del mismo caso. Los valores de P(NO APTA), scores de nivel, distancia Mahalanobis y recomendacion deben coincidir.

16. Como demostrarlo ante el profesor

16.1 Demostracion de 8 a 10 minutos

10. Mostrar la carpeta del proyecto y la separacion analysis/recognition/optimization/application.
11. Abrir el YAML smoke o camus10 y explicar que contiene las decisiónes del experimento.
12. Iniciar el localhost y mostrar que se ejecuta en 127.0.0.1.
13. Seleccionar un paciente-vista desde el manifest.
14. Mostrar ED y ES con las máscaras LV/MYO/LA.
15. Ejecutar y explicar Score G, Score M, Score T y P(NO APTA).
16. Mostrar Good Reference, nivel dominante y desviaciones robustas.
17. Mostrar el procesamiento recomendado y comparar identity con candidatos.

18. Descargar el ZIP del caso y mostrar la trazabilidad.
19. Cerrar indicando que la app reutiliza artefactos entrenados y no reentrena por imagen.

16.2 Preguntas probables y respuestas

Pregunta	Respuesta defendible
Por que Streamlit?	Permite una interfaz local reproducible con Python y separa la demostracion del nucleo científico. No reemplaza al paquete ni a las pruebas.
Por que necesita ED y ES?	El gate fue entrenado con agregacion paciente-vista mean + abs_delta. Usar una sola fase cambiaria la representación de entrada.
Por que necesita máscara?	Anatomia y textura se definen sobre LV/MYO/LA. Sin ROI, las mediciónes no responden a la misma pregunta científica.
La app usa la base para entrenar cada caso?	No. El entrenamiento es offline; la app carga gate y Good References ya estimados.
Que pasa si el filtro no mejora?	Identity es el control. Si ningun candidato valido supera la mejora minima, se mantiene la original.
Puede usarse clínicamente?	No. Es un prototipo academico que requiere validación externa, segmentacion validada, control de privacidad y aprobacion regulatoria.

17. Limitaciones actuales

- El modelo depende de la calidad y disponibilidad de las máscaras CAMUS o de una segmentacion externa validada.
- El gate actual fue definido para 2CH/4CH y ED/ES; otras vistas requieren reentrenamiento y referencias propias.
- Las matrices radiomics custom no se declaran equivalentes a IBSI o PyRadiomics sin validación numerica.
- La interfaz no implementa autenticacion, auditoria multiusuario ni almacenamiento clínico.
- Los scores son probabilidades del modelo, no probabilidades clínicas de diagnóstico o pronostico.
- Una imagen individual se considera exploratoria si no existe la fase complementaria.

18. Trabajo futuro

- Comparar inferencia de una sola fase contra el modelo paciente-vista mediante un estudio dedicado.
- Integrar una red de segmentacion validada para imágenes externas sin máscara.
- Agregar FastAPI como capa de servicio si se requiere integracion con otros sistemas.
- Implementar autenticacion y control de acceso antes de exponer el servidor en red.
- Realizar validación externa, calibracion y análisis de decisión clínica.
- Automatizar reportes PDF por caso con versión de modelo, hash de configuración y trazabilidad completa.

19. Conclusion

El localhost no es solamente una pantalla para cargar imágenes. Es una capa de operacion que conecta el sistema experimental con una demostracion reproducible: mantiene la separacion entre entrenamiento e inferencia, respeta las tres representaciónes, reutiliza los artefactos del nested CV, incorpora Good References y condiciona el procesamiento al diagnóstico.

Para la defensa, el aporte principal puede expresarse asi: EchoQuality 2.0 transforma una imagen ecocardiográfica en tres patrones independientes, fusióna sus evidencias para decidir aptitud, explica

la desviacion por nivel y evalua procesamiento solamente cuando puede mejorar la tarea sin comprometer anatomia o textura.

20. Referencias técnicas

- Gonzalez, R. C., y Woods, R. E. Digital Image Processing, 4th Edition. Fundamentos de muestreo, cuantizacion, filtrado, segmentacion, morfología y descripcion de patrones.
- Reporte de análisis de imágenes ecocardiográficas del proyecto EchoQuality. Recomendaciones sobre dos ramas, textura, reproducibilidad y diccionario de features.
- Streamlit Documentation. Run your app; st.file_uploader; caching and state; config.toml. https://docs.streamlit.io/
- GitHub Docs. Adding repositories, branches, commits and push with GitHub Desktop. https://docs.github.com/en/desktop
- Visual Studio Code Documentation. Python environments and interpreter selection. https://code.visualstudio.com/docs/python/environments
- scikit-learn Documentation. Pipelines, LogisticRegression, RandomForestClassifier, StratifiedGroupKFold, SelectKBest and LedoitWolf.

Anexo A. Comandos esenciales

```
conda activate echoquality-v2
python -m pip install -e ".[app]"
pytest
python run_experiment.py --config configs\experiments\smoke.yaml --force --fast
python scripts\13_validate_localhost.py --experiment-root results\smoke --config
configs\experiments\smoke.yaml
python run_streamlit.py
```

Anexo B. Archivos principales agregados

```
app/streamlit_app.py
src/echoquality/application/inference.py
src/echoquality/application/__init__.py
.streamlit/config.toml
scripts/13_validate_localhost.py
start_localhost.bat
start_localhost.ps1
docs/LOCALHOST_SETUP.md
```